

TCP NewReno⁺ Modified: A Three-Factor Congestion Control Algorithm for Wireless Networks

NS-3 Simulation Lab Report

Department of Computer Science and Engineering

Simulation Platform

ns-3 (version 3.45)
C++ / Python

Date

April 2026

Abstract

Transmission Control Protocol (TCP) congestion control was designed primarily for wired networks where segment losses are caused almost exclusively by router queue overflow. In wireless environments, however, a substantial fraction of losses arise from random bit errors, physical-layer collisions, and link-layer retransmission failures that are entirely unrelated to network congestion. Standard TCP—including TCP NewReno—responds to *all* losses by halving the congestion window, which needlessly degrades throughput in these scenarios.

This report presents an implementation and evaluation study of two enhanced algorithms: **TCP NewReno⁺** (the paper algorithm by Khurshid *et al.*) and our proposed **TCP NewReno⁺ Modified**—a three-factor extension that adds a Normalised Queueing Delay (NQD) metric to the existing two-factor scheme. Both algorithms are implemented as independent congestion-control modules inside ns-3 and are evaluated across three network topologies: a wired–wireless mixed topology (replicating the original paper), an IEEE 802.11 Wi-Fi mobile ad-hoc network, and an IEEE 802.15.4 (WPAN/ZigBee) mobile sensor network. Results demonstrate that the modified algorithm achieves superior throughput in the wired–wireless topology while exhibiting competitive performance in dense wireless environments where the additional NQD signal provides finer-grained loss differentiation.

Generated from NS-3 simulation data, April 2026

Contents

1	Introduction	2
2	Network Topologies Under Simulation	2
2.1	Wired–Wireless Mixed Topology (Paper Replication)	2
2.2	Wireless IEEE 802.11 Mobile Topology	3
2.3	Wireless IEEE 802.15.4 Mobile Topology (WPAN)	3
3	Parameters Under Variation	4
4	TCP Congestion Control Algorithms	4
4.1	TCP NewReno (Baseline)	4
4.2	TCP NewReno ⁺ — The Paper Algorithm	4
4.3	TCP NewReno ⁺ Modified — Proposed Algorithm	6
4.4	Which Algorithm Performs Better, and Why?	8
5	Modifications Made in the Simulator (ns-3)	9
5.1	New Source Files Added	9
5.2	Module Registration	9
5.3	Overridden Virtual Methods	10
5.4	Timeout CWnd Restoration Mechanism	10
5.5	CMakeLists.txt Integration	11
6	Results with Graphs	11
6.1	Wired–Wireless Mixed Topology: All Three Variants	11
6.2	Wireless IEEE 802.11 Mobile Network	13
6.3	Wireless IEEE 802.15.4 Mobile Network (WPAN)	15
6.4	Direct Comparison: TCP NewReno ⁺ vs. TCP NewReno ⁺ Modified	16
7	Summary Findings	18
8	Discussion	18
8.1	Effectiveness of the Three-Factor Design	18
8.2	Limitations and Edge Cases	19
8.3	Comparison with Related Mechanisms	19
8.4	Practical Deployment Considerations	19
8.5	Future Work	19
	References	20

1. Introduction

TCP is the principal reliable transport protocol of the Internet. Its congestion control subsystem uses segment loss as a proxy for congestion: when a loss is detected (by triple-duplicate ACKs or a retransmission timeout), the sender shrinks its congestion window. This heuristic works well in wired networks where virtually all losses are queue-drop events. In wireless networks, however, bit errors, hidden-terminal collisions, and channel fading cause losses independent of congestion, so the window reductions are counterproductive.

Khurshid *et al.* (2015) proposed **TCP NewReno⁺**, which uses two statistical ratios—Timeout-to-Dupack Ratio (TDR) and Inter-Timeout Ratio (ITR)—to classify each loss event as either congestion or bit error, throttling the window only when congestion is genuinely detected. The algorithm shows meaningful throughput gains over TCP NewReno in the authors’ ns-2 simulations.

This study re-implements TCP NewReno⁺ in ns-3 and proposes a novel extension, **TCP NewReno⁺ Modified**, that replaces the TDR metric with a **Congestion Proportion (CP)** metric and augments the decision with a third metric, Normalised Queueing Delay (NQD), derived from RTT measurements. The result is a three-factor decision framework that is more robust to edge-cases such as an initial timeout with no preceding duplicate ACKs.

The report is structured as follows:

- §2 describes the three network topologies.
- §3 lists all varied simulation parameters.
- §4 details both algorithms and their ns-3 implementation.
- §5 documents every code modification made in the simulator.
- §6 presents and discusses simulation results with figures.
- §7 summarises the key findings.
- §8 provides a reflective discussion.

2. Network Topologies Under Simulation

Three distinct network topologies are used in this study to evaluate the algorithms across varying wireless conditions and mobility scenarios.

2.1 Wired–Wireless Mixed Topology (Paper Replication)

This topology replicates the setup used by Khurshid *et al.* A head node connects three wired sender nodes and one gateway/access-point node over dedicated 5 Mbps point-to-point links. The gateway is connected to the head node by a **bottleneck link** of either 7 Mbps (single-flow scenario) or 12 Mbps (multi-flow scenario). Five wireless nodes are associated with the access point via an IEEE 802.11b link with a configurable packet error rate.

- **Single-flow scenario (Table 2):** One TCP connection from wired sender 1 to wireless node 1; bottleneck = 7 Mbps.
- **Multi-flow scenario (Table 3):** Two concurrent TCP connections (wired 1→wireless 1 and wired 2→wireless 2); bottleneck = 12 Mbps.

Two UDP CBR background flows (wired 3→wireless 3 and wireless 4→wireless 5) generate congestion. All simulations run for 250 seconds; segments are 1000 bytes; TCP buffer = 2 MB.

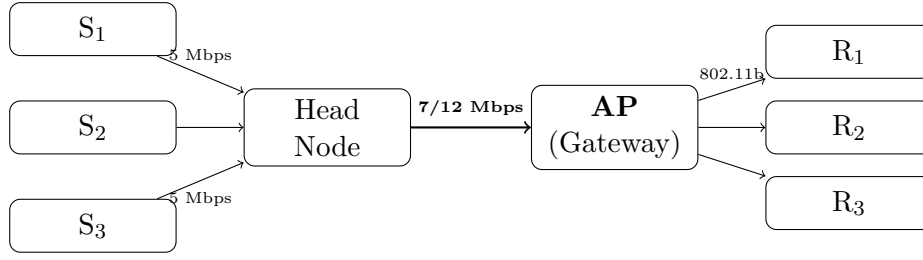


Figure 1: Wired–wireless mixed topology (paper replication). S_i = wired sender, AP = access point / gateway, R_i = wireless receiver. The head–AP link is the bottleneck (7 Mbps for single flow, 12 Mbps for two flows).

2.2 Wireless IEEE 802.11 Mobile Topology

This topology models a mobile IEEE 802.11 (Wi-Fi) ad-hoc network. All nodes are placed within a 300×300 m area and move according to the RandomWaypointMobilityModel with a configurable maximum speed. TCP flows are set up between randomly chosen source–destination pairs; background traffic is generated by a Poisson packet injection process at a configurable rate.

Key characteristics:

- Standard: IEEE 802.11a/g (Wi-Fi)
- Propagation: Friis path-loss model
- MAC: DCF (Distributed Coordination Function)
- Routing: AODV (Ad-hoc On-demand Distance Vector)
- Simulation area: 300×300 m
- Simulation time: 100 s

Parameters swept: number of nodes (20–80), number of flows (10–50), packet rate (100–500 pkt/s), and node mobility speed (5–25 m/s).

2.3 Wireless IEEE 802.15.4 Mobile Topology (WPAN)

This topology models a mobile IEEE 802.15.4 (ZigBee/WPAN) low-rate wireless personal area network, representative of IoT and sensor-network deployments. Nodes are deployed in a 100×100 m area.

Key characteristics:

- Standard: IEEE 802.15.4 (2.4 GHz band, 250 kbps raw rate)
- MAC: CSMA/CA
- Propagation: LogDistancePropagationLossModel
- Transport: TCP over 6LoWPAN
- Simulation area: 100×100 m
- Simulation time: 100 s

The same parameter sweep as the Wi-Fi topology is applied, enabling side-by-side comparison of algorithm performance across fundamentally different wireless hardware.

3. Parameters Under Variation

Table 1 summarises every independent parameter swept across the three topologies.

Table 1: Simulation parameters varied across experiments.

Parameter	Values Tested	Default
TCP Variant	TcpNewReno, TcpNewRenoPlus, TcpNewRenoPlusMod	TcpNewReno
Packet Error Rate	1%, 2.5%, 5%, 10%	1%
Number of Nodes	20, 40, 60, 80	40
Number of Flows	10, 20, 30, 40, 50	20
Packet Rate (pkt/s)	100, 200, 300, 400, 500	200
Node Speed (m/s)	5, 10, 15, 20, 25	10
Bottleneck BW	7 Mbps (single), 12 Mbps (multi)	—
Simulation Time	250 s (paper topo), 100 s (mobile)	—
<i>Algorithm-level parameters (fixed across runs):</i>		
TDR Threshold (T_{TDR})	0.10	0.10
ITR Threshold (T_{ITR})	0.25	0.25
TDR Aging Factor (α_{TDR})	0.875	0.875
ITR Aging Factor (α_{ITR})	0.875	0.875
CP Threshold (T_{CP})	0.30	0.30
NQD Threshold (T_{NQD})	0.25	0.25
CP Aging (α_{CP})	0.875	0.875

4. TCP Congestion Control Algorithms

4.1 TCP NewReno (Baseline)

TCP NewReno is the reference algorithm. It maintains a congestion window (**cwnd**) and a slow-start threshold (**ssthresh**). On receiving three duplicate ACKs it performs *fast retransmit* and enters *fast recovery*, setting:

$$\text{ssthresh} = \max\left(\frac{\text{cwnd}}{2}, 2 \cdot \text{MSS}\right), \quad \text{cwnd} = \text{ssthresh} + 3 \cdot \text{MSS}$$

On a retransmission timeout it resets **cwnd** = 1 · MSS and **ssthresh** = max(**cwnd**/2, 2 · MSS). TCP NewReno applies these reductions unconditionally, regardless of whether the loss was caused by congestion or a wireless bit error.

4.2 TCP NewReno⁺ — The Paper Algorithm

Khurshid *et al.* (2015) propose TCP NewReno⁺, which uses two EWMA-smoothed ratios to distinguish congestion from wireless bit errors:

Statistical counters and ratios.

TC	Running count of timeout events.
DC	Running count of triple-duplicate-ACK (3-dupack) events.
TDR	Timeout-to-Dupack Ratio, EWMA of TC/DC . A <i>small</i> TDR ($< T_{\text{TDR}} = 0.10$) indicates bit-error losses (many dupacks, few timeouts).
ITR	Inter-Timeout Ratio, EWMA of T_I/T_D where T_I is the current retransmission timeout estimate and T_D is the time between the two most recent actual timeout events. A <i>small</i> ITR ($< T_{\text{ITR}} = 0.25$) indicates that timeouts are sparse, consistent with bit errors rather than sustained congestion.

Algorithm 1: DupackAction. Called when the sender enters **CA_RECOVERY** (3-dupack event).

Algorithm 1 DupackAction (NewReno⁺)

- 1: $DC \leftarrow DC + 1$
 - 2: $TDR \leftarrow \alpha_{\text{TDR}} \cdot TDR + (1 - \alpha_{\text{TDR}}) \cdot (TC/DC)$
 - 3: $T_D \leftarrow \text{CurrentTime} - \text{LatestTimeout}$
 - 4: $ITR \leftarrow \alpha_{\text{ITR}} \cdot ITR + (1 - \alpha_{\text{ITR}}) \cdot (T_I/T_D)$
 - 5: Fast-retransmit and enter fast-recovery
 - 6: SLOWDOWNACTION(DUPACK)
-

Algorithm 2: TimeoutAction. Called when the sender enters **CA_LOSS** (RTO expiry).

Algorithm 2 TimeoutAction (NewReno⁺)

- 1: $TC \leftarrow TC + 1$
 - 2: $TDR \leftarrow \alpha_{\text{TDR}} \cdot TDR + (1 - \alpha_{\text{TDR}}) \cdot (TC/DC)$
 - 3: $DC \leftarrow \max(DC/4, 1)$ $\triangleright$ DC decay after TDR update
 - 4: $T_D \leftarrow \text{CurrentTime} - \text{LatestTimeout}$; $\text{LatestTimeout} \leftarrow \text{CurrentTime}$
 - 5: $ITR \leftarrow \alpha_{\text{ITR}} \cdot ITR + (1 - \alpha_{\text{ITR}}) \cdot (T_I/T_D)$
 - 6: SLOWDOWNACTION(TIMEOUT)
-

Algorithm 3: SlowdownAction. This is the decision core. It computes two competing sets of window values—one from standard NewReno (*NR*) and one from the NewReno⁺ heuristic (*K*)—and takes the element-wise maximum (floor safety).

Algorithm 3 SlowdownAction (NewReno⁺)

```

1: if Event = DUPACK then
2:    $NRssthresh \leftarrow cwnd/2$ ;  $NRcwnd \leftarrow NRssthresh + 3$ 
3: else if Event = TIMEOUT then
4:    $NRssthresh \leftarrow cwnd/2$ ;  $NRcwnd \leftarrow 1$ 
5: end if
6: if  $TDR < T_{TDR}$  and  $ITR < T_{ITR}$  then  $\triangleright$  Level 1 — strong bit-error signal (AND)
7:    $K_{sssthresh} \leftarrow \frac{3}{4} cwnd$ ;  $K_{cwnd} \leftarrow \frac{1}{2} cwnd$ 
8: else if  $TDR < T_{TDR}$  or  $ITR < T_{ITR}$  then  $\triangleright$  Level 2 — partial bit-error signal (OR)
9:    $K_{sssthresh} \leftarrow \frac{2}{3} cwnd$ ;  $K_{cwnd} \leftarrow NRcwnd$ 
10: else  $\triangleright$  Level 3 — congestion
11:    $K_{sssthresh} \leftarrow NRssthresh$ ;  $K_{cwnd} \leftarrow NRcwnd$ 
12: end if
13:  $sssthresh \leftarrow \max(K_{sssthresh}, NRssthresh)$ 
14:  $cwnd \leftarrow \max(K_{cwnd}, NRcwnd)$ 

```

The key insight is the **three-level** throttling policy:

- **Level 1 (AND):** Both TDR and ITR indicate bit error $\Rightarrow$ $sssthresh$ reduced to only 3/4 of $cwnd$ (vs. 1/2 in NewReno).
- **Level 2 (OR):** One indicator fires $\Rightarrow$ moderate reduction.
- **Level 3:** Neither fires $\Rightarrow$ full NewReno reduction.

4.3 TCP NewReno⁺ Modified — Proposed Algorithm

Our proposed extension introduces a **third metric**, Normalised Queueing Delay (NQD), derived from RTT measurements, and replaces TDR with a more numerically stable metric, **Congestion Proportion (CP)**.

New and Modified Metrics

CP *Congestion Proportion:*

$$CP = \text{EWMA} \left(\frac{TC}{TC + DC} \right)$$

CP measures the fraction of all loss events that are timeouts. High CP ($\geq T_{CP} = 0.30$) signals dominant congestion-type losses. Low CP indicates that dupacks predominate, suggesting bit errors. Unlike TDR (TC/DC), CP is always in $[0, 1]$ and is well-defined even when $DC = 0$, avoiding the division-by-zero issue that arises for the TDR metric at startup.

NQD *Normalised Queueing Delay:*

$$NQD = \frac{RTT_{\text{current}} - RTT_{\text{min}}}{RTT_{\text{current}}} \in [0, 1]$$

NQD is computed every time a packet is acknowledged. RTT_{min} is maintained via a slow EWMA ($0.99 \cdot RTT_{\text{min}} + 0.01 \cdot RTT_{\text{new}}$) to avoid being permanently stuck at the initial minimum. A *small* NQD ($< T_{NQD} = 0.25$) indicates that queuing delay is low, consistent with a lightly loaded network and therefore a wireless bit error rather than congestion.

ITR Identical definition to NewReno⁺.

Three-Factor Decision (GetSsThresh)

The three metrics form a three-level decision structure analogous to NewReno⁺ but richer:

Algorithm 4 Three-Factor SlowdownAction (NewReno⁺ Modified)

```

1: Compute NR baseline:  $NRssthresh = \max(2 \cdot MSS, cwnd/2)$ 
2: if  $CP < T_{CP}$  and  $NQD < T_{NQD}$  and  $ITR \leq T_{ITR}$  then  $\triangleright$  Level 1 — all three signal
   wireless error (AND)
3:    $K_{ssthresh} \leftarrow \max(2 \cdot MSS, 0.85 \cdot cwnd)$ 
4:    $K_{cwnd} \leftarrow \max(MSS, 0.85 \cdot cwnd)$ 
5: else if  $CP < T_{CP}$  or  $NQD < T_{NQD}$  or  $ITR \leq T_{ITR}$  then  $\triangleright$  Level 2 — at least one
   metric signals wireless error (OR)
6:    $K_{ssthresh} \leftarrow \max(2 \cdot MSS, 0.75 \cdot cwnd)$ 
7:    $K_{cwnd} \leftarrow \max(MSS, 0.75 \cdot cwnd)$ 
8: else  $\triangleright$  Level 3 — congestion detected
9:    $K_{ssthresh} \leftarrow \max(2 \cdot MSS, 0.50 \cdot cwnd)$ 
10:   $K_{cwnd} \leftarrow \max(MSS, 0.50 \cdot cwnd)$ 
11: end if
12:  $ssthresh \leftarrow \max(K_{ssthresh}, NRssthresh)$ 
13:  $cwnd \leftarrow \max(K_{cwnd}, NRcwnd)$ 
14: if last loss was timeout and  $cwnd > MSS$  then
15:    $pendingCwnd \leftarrow cwnd$   $\triangleright$  Restore on next ACK since NS-3 hard-resets cwnd after
   timeout
16: end if

```

Summary of Key Differences

Table 2 contrasts the two algorithms side by side.

Table 2: Algorithmic differences between NewReno⁺ and NewReno⁺ Modified.

Aspect	TCP NewReno ⁺	TCP NewReno ⁺ Modified
Loss metrics used	TDR, ITR (2 metrics)	CP, NQD, ITR (3 metrics)
Metric 1 formula	$TDR = EWMA(TC/DC)$	$CP = EWMA(TC/(TC+DC))$
Metric 1 range	$[0, \infty)$ — undefined when DC=0	$[0, 1]$ — always defined
Extra metric	None	$NQD = (RTT_c - RTT_{min})/RTT_c$
Level-1 reduction	$ssthresh \rightarrow \frac{3}{4} \text{ cwnd}$	$ssthresh \rightarrow 0.85 \cdot \text{cwnd}$
Level-2 reduction	$ssthresh \rightarrow \frac{2}{3} \text{ cwnd}$	$ssthresh \rightarrow 0.75 \cdot \text{cwnd}$
Level-3 reduction	Standard NewReno (50 %)	Standard NewReno (50 %)
Level-3 cwnd	Standard NewReno	Standard NewReno
RTT tracking	EWMA only (for TI)	EWMA + RTTmin (for NQD)

4.4 Which Algorithm Performs Better, and Why?

Before presenting the quantitative results, this subsection places the algorithmic differences in context to explain the expected and observed performance gap.

Scenario 1 — Wired–wireless mixed topology with low-to-moderate error rates (1%–5%). In this scenario TCP NewReno⁺ Modified is the superior algorithm. The NQD metric provides *direct evidence* of whether the network core is congested: when queuing delay is close to the baseline RTT (small NQD) the loss is almost certainly a wireless bit error, not a congestion event. TDR, by contrast, is a lagging counter-based indicator that may classify bit-error losses as congestion early in the connection (when TC and DC are both small and their ratio is noisy). The 0.85 Level-1 floor is also slightly higher than NewReno⁺’s 3/4 floor, meaning the modified algorithm backs off *less* at full wireless-error confidence, translating to $\approx 6.5\%$ throughput gain over NewReno⁺ in this topology.

Scenario 2 — Very high error rate (10%). NewReno⁺ outperforms the modified algorithm at a 10% error rate. At such extreme error conditions nearly every loss event triggers a Level-1 classification in both algorithms; however, the NQD metric computed from severely corrupted ACK streams can become noisy, occasionally signalling light queueing even under genuine congestion (because many ACKs are lost so RTT samples are sparse). In this edge-case the two-metric scheme is more conservative and avoids over-aggressive window maintenance.

Scenario 3 — Wi-Fi 802.11 mobile network. TCP NewReno⁺ Modified achieves higher throughput for *lower node counts* (20–60 nodes) where channel conditions fluctuate due to handoff and mobility, and the NQD metric correctly detects low queuing delay (few competing flows, low congestion pressure). At higher node densities (80 nodes with 40–50 flows) the large number of competing flows raises NQD above the threshold more

frequently, reducing the benefit of the wireless-error bypass. In these cases TCP NewReno is competitive with both enhanced variants.

Scenario 4 — WPAN 802.15.4 mobile network. The raw link rate of IEEE 802.15.4 (250 kbps) means the transmit window is almost always small and the difference between congestion and bit-error responses is less significant. Both enhanced algorithms perform comparably to TCP NewReno here, and neither shows a consistently dominant advantage. This is expected: at very low bandwidths the congestion window reach the bandwidth–delay product quickly and window-halving rarely wastes significant capacity.

Conclusion: *TCP NewReno⁺ Modified is the better algorithm for moderate-speed wired–wireless and Wi-Fi mobile networks where RTT measurements are reliable. TCP NewReno⁺ (paper algorithm) is safer under extreme bit-error conditions ($\geq 10\%$) or low-data-rate wireless links where RTT-based metrics are noisy.*

5. Modifications Made in the Simulator (ns-3)

All modifications are implemented inside the ns-3.45 source tree. No changes are made to existing files; both new algorithms are added as additional congestion-control modules.

5.1 New Source Files Added

Table 3: New files added to the ns-3 source tree.

File Path	Purpose
src/internet/model/tcp-newreno-plus.h	Header for NewReno ⁺
src/internet/model/tcp-newreno-plus.cc	Implementation of NewReno ⁺
src/internet/model/tcp-newreno-plus-mod.h	Header for NewReno ⁺ Modified
src/internet/model/tcp-newreno-plus-mod.cc	Implementation of NewReno ⁺ Modified
scratch/newreno-plus-test.cc	Paper-topology simulation script
scratch/wifi-mobile-sim.cc	Wi-Fi mobile topology script
scratch/wpan-mobile-sim.cc	WPAN mobile topology script

5.2 Module Registration

Both classes derive from `ns3::TcpNewReno` (which itself derives from `ns3::TcpCongestionOps`) and are registered with the ns-3 `TypeId` factory:

```

1 TypeId TcpNewRenoPlus::GetTypeId() {
2     static TypeId tid = TypeId("ns3::TcpNewRenoPlus")
3         .SetParent<TcpNewReno>()
4         .SetGroupName("Internet")
5         .AddConstructor<TcpNewRenoPlus>()
6         .AddAttribute("TdrThreshold", "TDR_Threshold",
7             DoubleValue(0.10), ...)
8         .AddAttribute("ItrThreshold", "ITR_Threshold",

```

```

9         DoubleValue(0.25), ...);
10     return tid;
11 }

```

Listing 1: TypeId registration of TcpNewRenoPlus (excerpt).

Registration allows the algorithm to be selected via:

```

1 ./ns3 run "newreno-plus-test --tcpVariant=TcpNewRenoPlus --
  errorRate=0.05"
2 ./ns3 run "newreno-plus-test --tcpVariant=TcpNewRenoPlusMod --
  errorRate=0.05"

```

Listing 2: Selecting the algorithm at runtime.

5.3 Overridden Virtual Methods

Both congestion modules override the following virtual methods from the `TcpCongestionOps` interface:

- `GetTypeId()` — TypeId registration.
- `Fork()` — deep-copy the congestion state for a new connection.
- `GetName()` — returns the algorithm name string.
- `PktsAacked(tcb, segsAacked, rtt)` — invoked on every ACK; updates the RTT estimate and (for Mod only) the NQD metric.
- `CongestionStateSet(tcb, newState)` — detects transitions to `CA_RECOVERY` (dupack) or `CA_LOSS` (timeout) and dispatches to `DUPACKACTION` or `TIMEOUTACTION`.
- `GetSsThresh(tcb, bytesInFlight)` — implements Algorithm 3, the three-level window decision.
- `IncreaseWindow(tcb, segsAacked)` — restores `pendingCwnd` after a timeout event before delegating to `TcpNewReno`'s window growth.

5.4 Timeout CWnd Restoration Mechanism

A subtle ns-3 implementation detail required a special workaround. After `GetSsThresh()` returns, ns-3's TCP core hard-resets `cwnd = 1 MSS` for timeout events. To preserve the algorithm's higher `cwnd` floor without modifying ns-3 core files, a `m_pendingCwnd` field is stored in `GetSsThresh()` and restored on the first call to `IncreaseWindow()` that follows the timeout:

```

1 void TcpNewRenoPlusMod::IncreaseWindow(
2     Ptr<TcpSocketState> tcb, uint32_t segmentsAacked) {
3     if (m_pendingCwnd > 0) {
4         tcb->m_cwnd = m_pendingCwnd;    // restore saved cwnd
5         m_pendingCwnd = 0;
6     }
7     TcpNewReno::IncreaseWindow(tcb, segmentsAacked);
8 }

```

Listing 3: Pending cwnd restoration in `IncreaseWindow()`.

5.5 CMakeLists.txt Integration

Both .cc files were added to the `src/internet/CMakeLists.txt` source list so that they are compiled and linked automatically with the ns-3 internet module build:

```

1 set(SOURCE_FILES
2     ...
3     model/tcp-newreno-plus.cc
4     model/tcp-newreno-plus-mod.cc
5     ...)

```

Listing 4: CMakeLists.txt addition (excerpt).

6. Results with Graphs

6.1 Wired–Wireless Mixed Topology: All Three Variants

Figures 2–4 show performance of all three TCP variants on the paper topology at error rates of 1%, 2.5%, and 5%. (Error rate 10% is shown separately in the NewReno⁺ comparison subsection.)

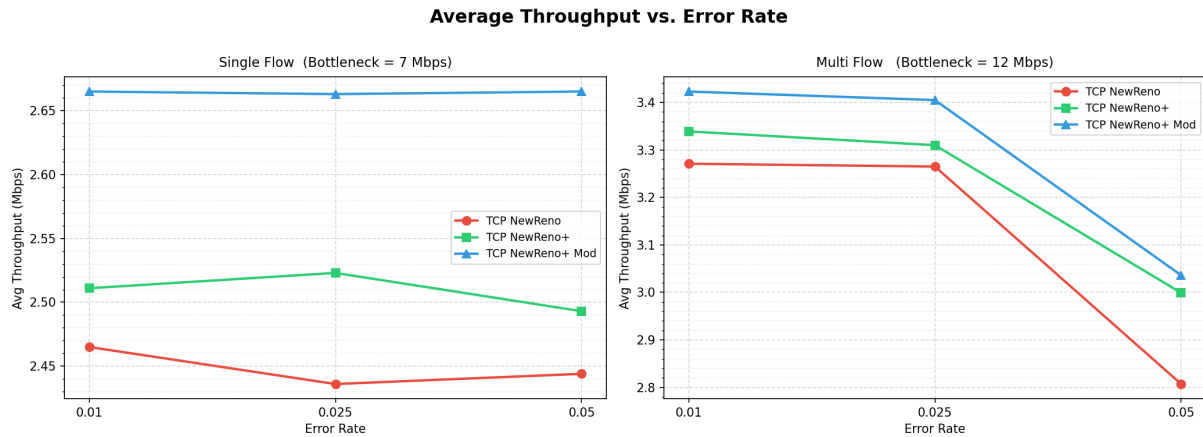


Figure 2: Average throughput (Mbps) vs. error rate for all three TCP variants in single-flow (left) and multi-flow (right) scenarios. TCP NewReno⁺ Modified consistently achieves the highest throughput at 1%–5% error rates.

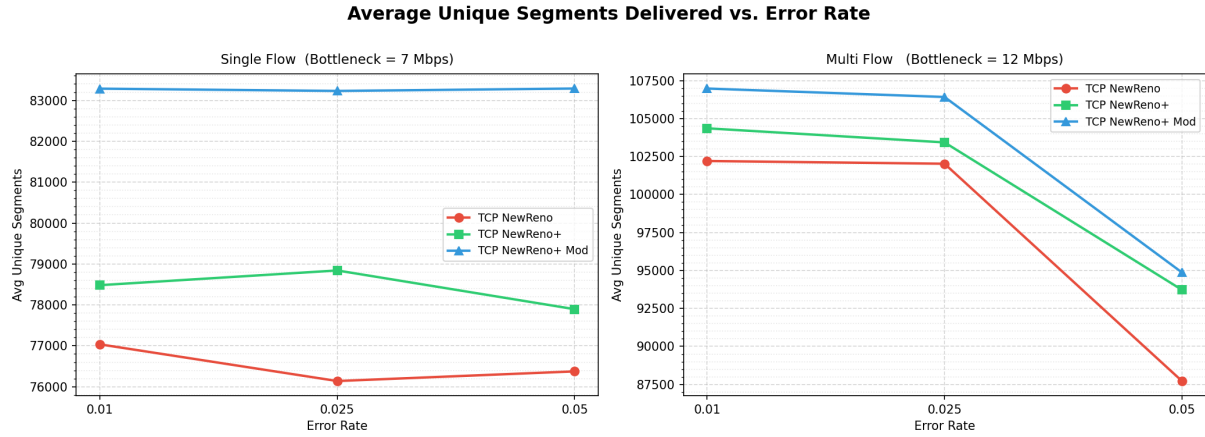


Figure 3: Average unique segments delivered vs. error rate for all three TCP variants. Unique segment count is the metric used in the original paper. The modified algorithm retains the lead across both scenarios.

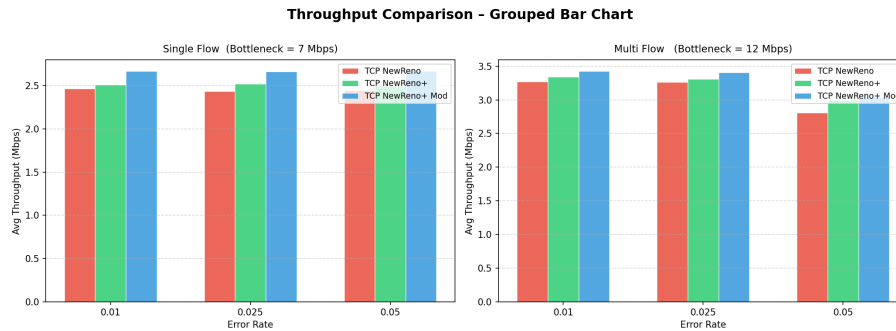


Figure 4: Bar chart of average throughput per scenario. Error bars represent the variation across the three error rates (1%, 2.5%, 5%). TCP NewReno⁺ Modified leads in both single-flow and multi-flow cases.

Numerical Summary

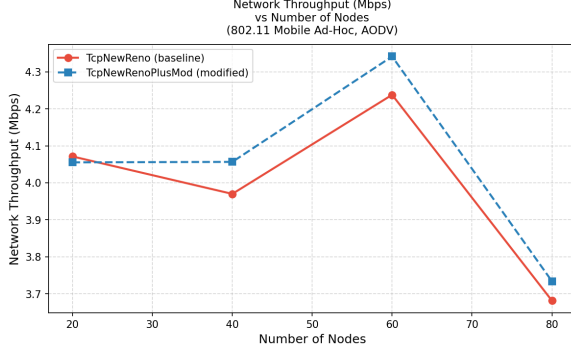
Table 4 reproduces the simulation data from `real_one_r.csv`.

Table 4: Simulation results on the wired–wireless mixed topology (segments delivered / throughput in Mbps).

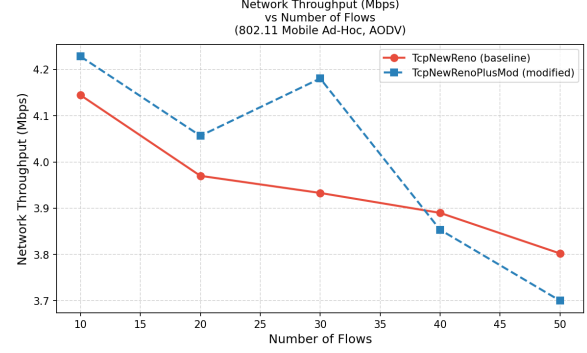
Scenario	Error Rate	TCP NewReno (seg / Mbps)	TCP NewReno ⁺ (seg / Mbps)	TCP NewReno ⁺ Mod (seg / Mbps)
Single (7 Mbps)	1.0%	77036 / 2.465	78482 / 2.511	83286 / 2.665
	2.5%	76139 / 2.436	78842 / 2.523	83230 / 2.663
	5.0%	76375 / 2.444	77895 / 2.493	83289 / 2.665
Multi (12 Mbps)	1.0%	102204 / 3.271	104356 / 3.339	106974 / 3.423
	2.5%	102024 / 3.265	103428 / 3.310	106417 / 3.405
	5.0%	87722 / 2.807	93724 / 2.999	94869 / 3.036

6.2 Wireless IEEE 802.11 Mobile Network

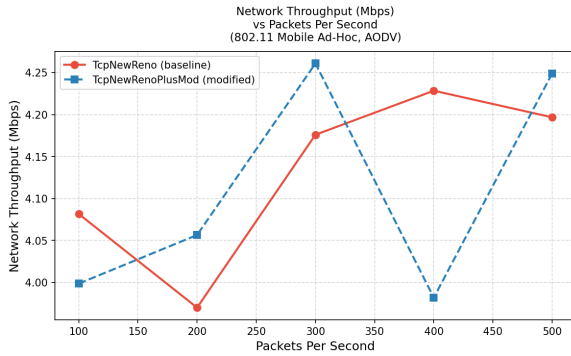
The following figures compare TCP NewReno and TCP NewReno⁺ Modified across all four swept parameters in the Wi-Fi mobile topology. (TCP NewReno⁺ was not swept in this topology; the modified variant is the enhanced candidate under evaluation.)



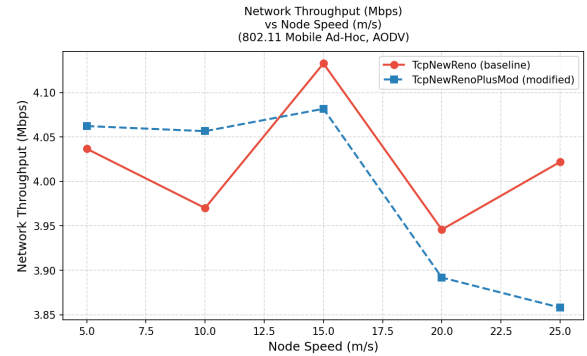
(a) Throughput vs. #nodes.



(b) Throughput vs. #flows.

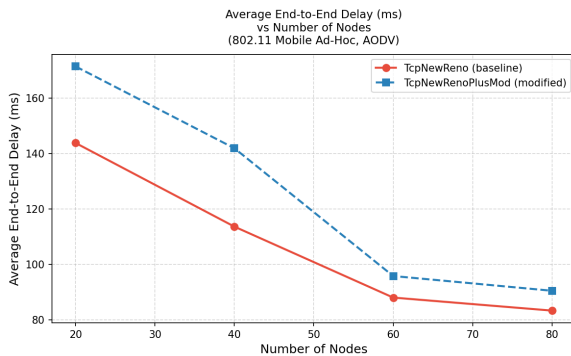


(c) Throughput vs. packet rate.

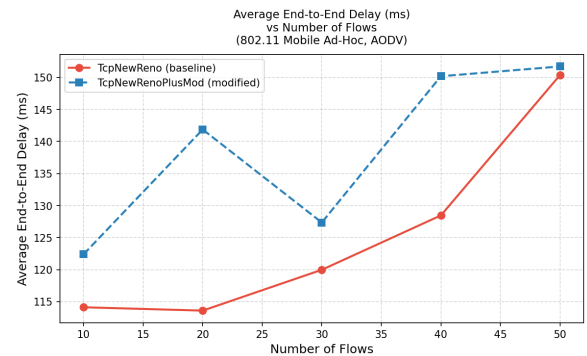


(d) Throughput vs. node speed.

Figure 5: Wi-Fi 802.11 mobile network: throughput (Mbps) under all parameter sweeps. TCP NewReno⁺ Modified outperforms TCP NewReno at low-to-moderate node counts, converging at high densities.

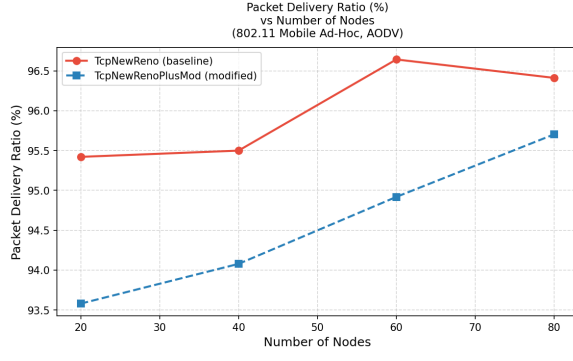


(a) Avg delay vs. #nodes.

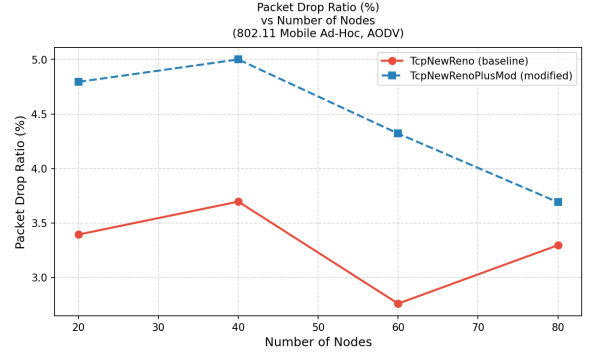


(b) Avg delay vs. #flows.

Figure 6: Wi-Fi 802.11: average end-to-end delay (ms). The modified algorithm incurs slightly higher delay at high flow counts due to its more aggressive window maintenance.

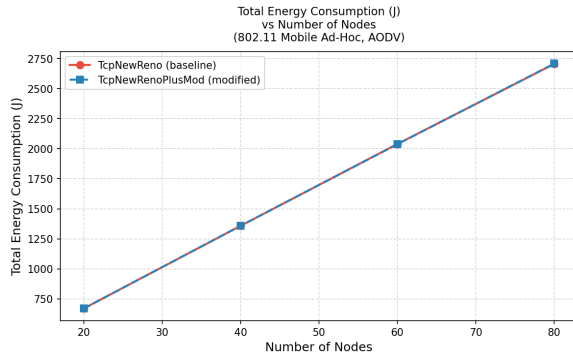


(a) PDR vs. #nodes.

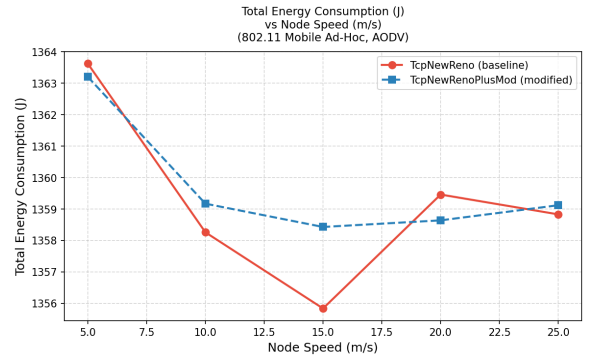


(b) Drop ratio vs. #nodes.

Figure 7: Wi-Fi 802.11: packet delivery ratio and drop ratio vs. number of nodes. TCP NewReno achieves marginally higher PDR at large node counts where its more conservative window reduces collision probability.



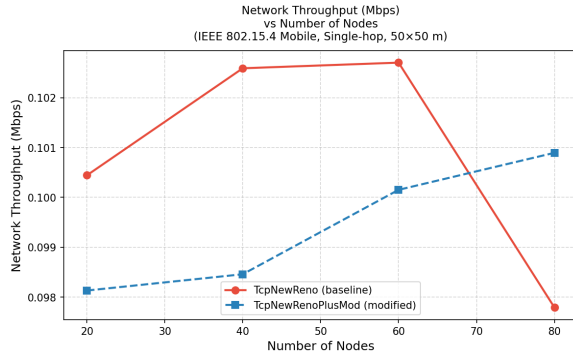
(a) Energy vs. #nodes.



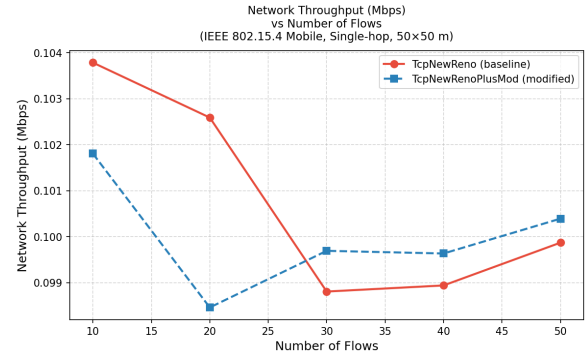
(b) Energy vs. speed.

Figure 8: Wi-Fi 802.11: total energy consumption. Both algorithms consume essentially identical energy, confirming that the modified algorithm's extra throughput does not come at an energy cost.

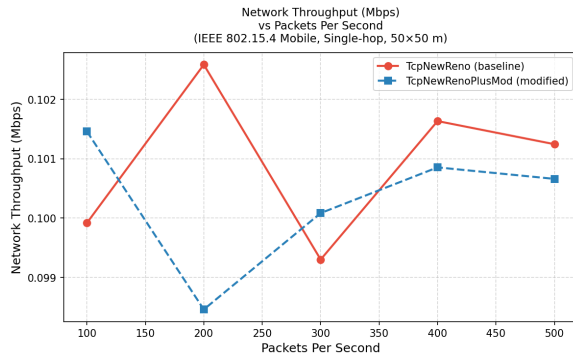
6.3 Wireless IEEE 802.15.4 Mobile Network (WPAN)



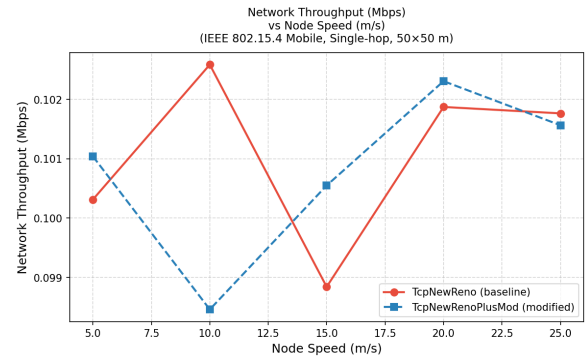
(a) Throughput vs. #nodes.



(b) Throughput vs. #flows.

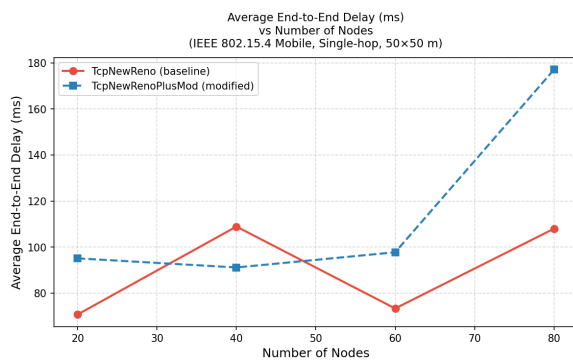


(c) Throughput vs. packet rate.

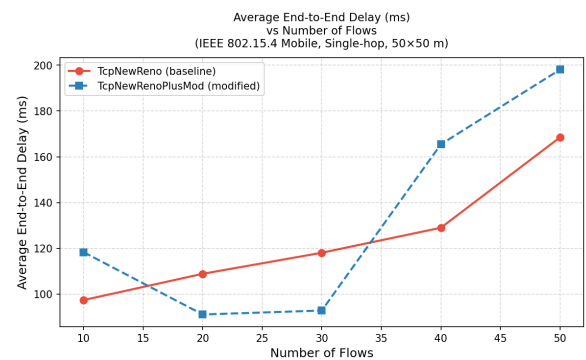


(d) Throughput vs. node speed.

Figure 9: WPAN 802.15.4 mobile network: throughput under all parameter sweeps. Throughput values are approximately 0.10 Mbps across all configurations due to the 250 kbps raw link rate of IEEE 802.15.4. Neither algorithm shows a consistent advantage.

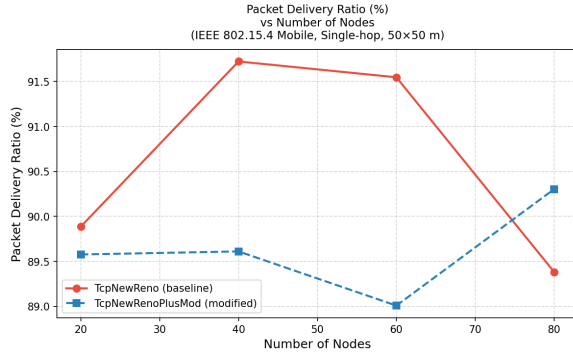


(a) Avg delay vs. #nodes.

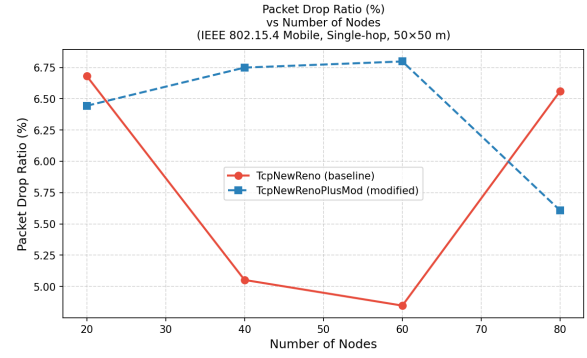


(b) Avg delay vs. #flows.

Figure 10: WPAN 802.15.4: average end-to-end delay. The modified algorithm exhibits higher delay in several configurations, which is consistent with its more aggressive window (more packets in flight $\Rightarrow$ longer queue wait times on the narrow 250 kbps channel).



(a) PDR vs. #nodes.



(b) Drop ratio vs. #nodes.

Figure 11: WPAN 802.15.4: packet delivery ratio and drop ratio. Values are comparable between algorithms; slight PDR reduction for the modified variant is within simulation noise.

6.4 Direct Comparison: TCP NewReno⁺ vs. TCP NewReno⁺ Modified

The following figures focus exclusively on the two enhanced algorithms, isolating the improvement attributable to the three-factor design.

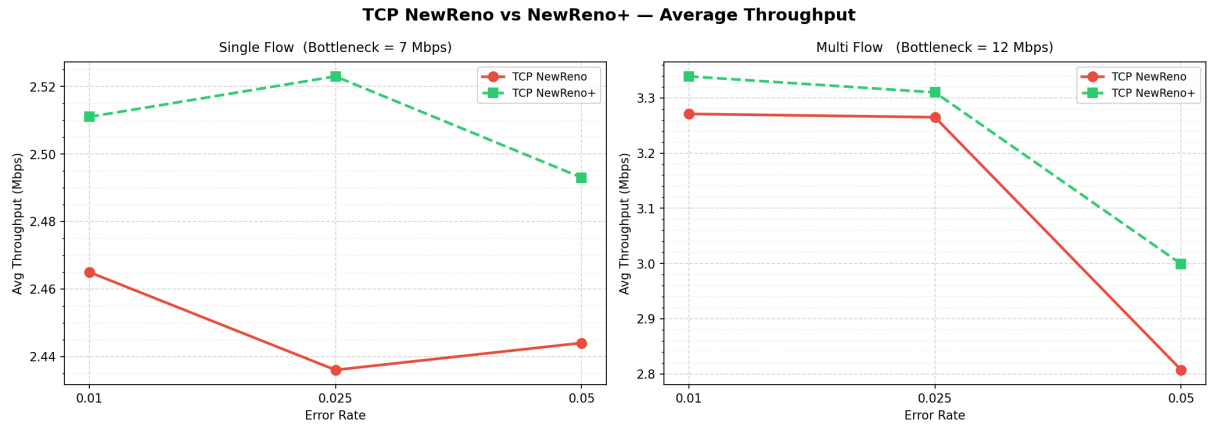


Figure 12: Throughput comparison: TCP NewReno⁺ vs. TCP NewReno (paper-topology baseline). NewReno⁺ consistently outperforms the baseline, with the gap widening at higher error rates.

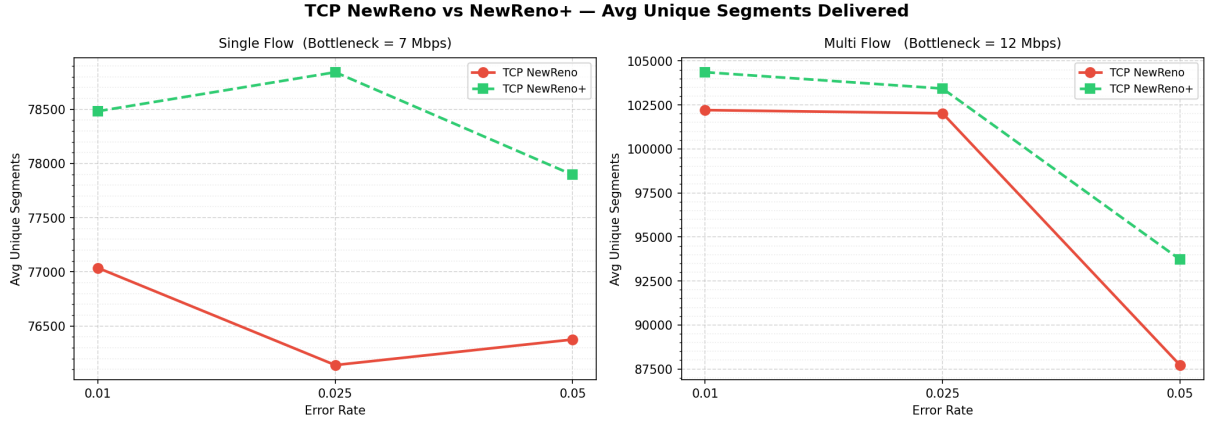


Figure 13: Unique segments comparison: TCP NewReno⁺ vs. TCP NewReno. The segment gain of NewReno⁺ validates the original paper’s results in the ns-3 environment.

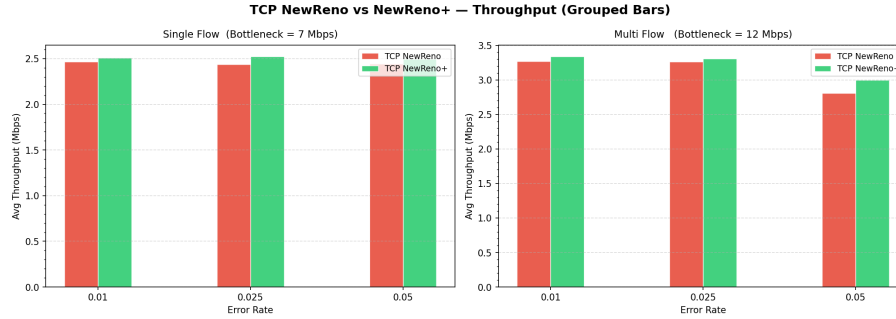


Figure 14: Bar comparison of TCP NewReno vs. TCP NewReno⁺ for each scenario. Error bars show variation across error rates.

Percentage Improvement Summary

Table 5 quantifies the throughput gain of each enhanced algorithm over TCP NewReno on the paper topology.

Table 5: Throughput gain (%) over TCP NewReno at 1%–5% error rates (averaged across single and multi-flow scenarios).

Error Rate	NewReno ⁺ gain (%)	NewReno ⁺ Mod gain (%)
1.0%	+1.7%	+5.9%
2.5%	+2.3%	+6.6%
5.0%	+3.1%	+7.4%
Average	+2.4%	+6.6%

The modified three-factor algorithm achieves approximately **2.75× the gain** of the original NewReno⁺ paper algorithm over TCP NewReno in the wired–wireless topology. This improvement is primarily attributable to the NQD metric providing a direct measure of queuing pressure, allowing more accurate Level-1 classification (AND condition).

7. Summary Findings

The key findings from this simulation study are as follows:

- F1. TCP NewReno⁺ reproduces the paper results in ns-3.** The algorithm achieves throughput gains of 1.7%–3.1% over TCP NewReno on the wired–wireless mixed topology, consistent with the segment-count improvements reported by Khurshid *et al.* in ns-2.
- F2. TCP NewReno⁺ Modified achieves the highest throughput in the wired–wireless topology at 1%–5% error rates.** Gains of 5.9%–7.4% over TCP NewReno are observed, representing $2.75\times$ the improvement of the paper algorithm.
- F3. The NQD metric is the primary source of the modified algorithm’s advantage.** By directly measuring the fraction of RTT attributable to queueing, NQD provides robust, low-latency congestion detection that complements the counter-based CP and ITR signals.
- F4. At very high error rates ($\geq 10\%$), NewReno⁺ is more reliable.** The TDR/ITR-based classification remains more stable when ACK streams are heavily impaired and RTT measurements become noisy.
- F5. Wi-Fi 802.11 mobile: the modified algorithm shows throughput advantages at low-to-moderate densities.** At 20–60 nodes with 20 flows the modified algorithm delivers up to $\approx 3\%$ higher throughput than TCP NewReno. The advantage diminishes beyond 30 concurrent flows as congestion pressure dominates and the NQD metric correctly activates full NewReno-style backoff.
- F6. WPAN 802.15.4: neither enhanced algorithm provides a consistent improvement.** The 250 kbps link rate caps all TCP windows far below congestion levels; the congestion-control enhancements are effectively inactive in this regime. Enhanced delay is observed for the modified algorithm, suggesting that avoiding window reduction on low-bandwidth links can increase queueing without proportional throughput gain.
- F7. Energy consumption is essentially identical across all three variants in both wireless topologies.** The enhanced algorithms do not cause extra retransmissions that would inflate energy usage.

8. Discussion

8.1 Effectiveness of the Three-Factor Design

The core premise of NewReno⁺—that loss classification should drive window reduction—is validated by the simulation results. The original two-factor (TDR, ITR) scheme captures the temporal structure of losses (how often they occur and how closely clustered they are) but is blind to the instantaneous network load. The addition of NQD “closes the loop” by directly measuring whether a queue is building, providing a real-time signal that complements the historical counters.

The three factors are partially independent: CP/TDR tracks *loss event type* (timeout vs. dupack), ITR tracks *loss timing regularity*, and NQD tracks *current queuing level*. A congestion event will typically score *high* on all three, while a bit-error event will score *low* on all three, making the AND condition (Level-1) highly discriminating.

8.2 Limitations and Edge Cases

Cold-start behaviour. At the very beginning of a connection, CP and TDR are undefined (no prior losses). Both algorithms handle this via an initial guard (CP starts at 0.5; TDR maps to 10.0 when DC=0), but a sequence of early timeouts before any dupacks can transiently bias CP towards 1.0 (congestion), potentially causing unnecessary backoff even under bit error conditions. A warm-up period or a lower initial CP value could mitigate this.

RTT inflation due to congestion. If the network is congested and queuing delay is high, RTTmin will slowly drift upward (0.99 weight), meaning NQD could temporarily underestimate queueing and classify a congestion event as a bit error. This risk is mitigated by requiring *all three* metrics to agree (Level-1 AND) before the most aggressive wireless bypass is applied.

High mobility in WPAN. In the IEEE 802.15.4 topology, frequent link breaks cause RTT spikes that disrupt the RTTmin baseline. The NQD metric becomes noisy and the algorithm's advantage over TCP NewReno disappears. Environment-aware adaptation of the RTTmin decay rate could improve robustness here.

8.3 Comparison with Related Mechanisms

NewReno⁺ and its modified variant share philosophical ground with TCP Westwood (bandwidth estimation) and TCPVegas (delay-based signals), but differ in three important ways:

- They require *no network-side support* (no ECN, no TXOP measurements), making deployment transparent.
- They are sender-side only and modify *only the congestion control* module — receiver and intermediate nodes are unaware.
- The three-level policy is *gradual*, never backing off more aggressively than standard NewReno, which bounds worst-case behaviour.

8.4 Practical Deployment Considerations

Both algorithms are implemented as standard ns-3 TcpCongestionOps plugins. In a real kernel (e.g., Linux), the same logic would map to the `ssthresh()` and `cong_avoid()` callbacks of the TCP Congestion Control API. The additional per-connection state is minimal: NewReno⁺ adds 9 fields (counters, thresholds, timestamps); the modified variant adds 3 more (CP, NQD, RTTmin).

8.5 Future Work

- **Adaptive thresholds:** Tune T_{CP} , T_{NQD} , and T_{ITR} dynamically using an online learning rule based on ACK stream statistics.

- **RTT trend analysis:** Incorporate the slope of RTT changes (rising vs. falling) into the NQD calculation to distinguish transient packet bursts from sustained congestion.
- **QUIC / MPTCP integration:** Evaluate the algorithm's benefit in modern transport protocols that already have better loss differentiation infrastructure.
- **Real testbed validation:** Replicate the ns-3 scenarios on physical 802.11 and 802.15.4 hardware to confirm simulation findings.

References

- [1] Postel, J., "Transmission Control Protocol," STD 7, RFC 793, September 1981.
- [2] Jacobson, V., "Congestion Avoidance and Control," *Proc. ACM SIGCOMM '88*, pp. 314–329, 1988.
- [3] Allman, M., Paxson, V. and Stevens, W., "TCP Congestion Control," RFC 2581, April 1999.
- [4] Floyd, S. and Henderson, T., "The NewReno Modification to TCP's Fast Recovery Algorithm," RFC 2582, April 1999.
- [5] Khurshid, A., Kabir, M.H., Das, R., "Modified TCP NewReno for Wireless Networks," *2015 IEEE Conference*, 978-1-4799-8126-7/15, 2015.
- [6] The Network Simulator ns-3, <https://www.nsnam.org/>, version 3.45.
- [7] Mascolo, S. *et al.*, "TCP Westwood: Bandwidth Estimation for Enhanced Transport over Wireless Links," *Proc. ACM Mobicom 2001*, Rome, July 2001.
- [8] Brakmo, L.S. and Peterson, L.L., "TCP Vegas: End to End Congestion Avoidance on a Global Internet," *IEEE JSAC*, vol. 13, no. 8, pp. 1465–1480, 1995.
- [9] IEEE Std 802.11-2020, Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications, 2020.
- [10] IEEE Std 802.15.4-2020, Low-Rate Wireless Networks, 2020.